

ESOF 322

Software Engineering

LECTURE: 7

J. L. Pach

OUTLINE:

- Agile Software Development

AGILE

Software Development

WHY AGILE METHODS EXIST — THE MOTIVATION 1/3

1. Business Context
   - Businesses operate in **rapidly changing global environments** — new markets, economic shifts, competing products
   - Software must be delivered **quickly** to exploit opportunities and respond to competition
   - Companies may **sacrifice quality or compromise on requirements** if they can deploy software fast
2. The Problem with Plan-Driven Approaches
3. Why Requirements Always Change

WHY AGILE METHODS EXIST — THE MOTIVATION 2/3

1. Business Context
2. The Problem with Plan-Driven Approaches
   - Traditional waterfall/specification-based processes require **complete, stable requirements upfront** — which is practically impossible in dynamic environments
   - When requirements change (and they always do), the design and implementation must be **reworked and retested**, making the process lengthy
   - By the time software is finally delivered, **the original business need may no longer exist**
   - Exception: **safety-critical systems** (e.g., aerospace, medical devices) genuinely require plan-driven approaches due to the need for exhaustive analysis

WHY AGILE METHODS EXIST — THE MOTIVATION 3/3

1. Business Context
2. The Problem with Plan-Driven Approaches
3. Why Requirements Always Change
   - Customers cannot predict how a system will affect working practices
   - They cannot foresee how it will interact with other systems
   - Real requirements often only become clear **after** users gain experience with a delivered system
   - External factors continuously drive requirement changes

WHAT AGILE METHODS ARE

Core Definition

Agile methods are **incremental development approaches** that:

- Produce software in **small, frequent increments** (typically every 2–3 weeks)
- **Interleave** specification, design, and implementation — no separate phases
- **Minimize documentation** — informal communication over formal written documents
- **Involve customers continuously** for rapid feedback on changing requirements
- Use **extensive tool support** (automated testing, CI tools, UI generation)

KEY HISTORICAL CONTEXT

- Agile emerged in the **late 1990s** as a reaction to heavyweight plan-driven methods
- Major agile methods:
  - **Extreme Programming (XP)** (1999), **Scrum** (2001), **DSDM** (2003)
- In the 1980s–early 1990s, the dominant view was: `better software = careful planning + formalized QA + rigorous processes`
- That approach was designed for `large, geographically dispersed teams` on long-lived government/aerospace systems — justified when:
  - Multiple teams must be coordinated; System is safety-critical; Many people will maintain software over its lifetime
- When applied to **small/medium business systems**, the overhead became disproportionate: more time spent on *how* to develop than on actual development.

THE AGILE MANIFESTO

Published at agilemanifesto.org:

Value (More Important)	Over (Less Important but Still Valued)
Individuals and interactions	Processes and tools
Working software	Comprehensive documentation
Customer collaboration	Contract negotiation
Responding to change	Following a plan

> Critical nuance: The manifesto does NOT say the right column has no value — it says the **left column is valued MORE**

FIVE CORE PRINCIPLES OF AGILE METHODS

Principle	Description
Customer involvement	Customers closely involved throughout; provide/prioritize requirements; evaluate iterations
Embrace change	Design to accommodate changing requirements as a natural expectation, not an exception
Incremental delivery	Software developed in increments; customer specifies requirements per increment
Maintain simplicity	Actively eliminate complexity from both software and process
People, not process	Exploit team skills; let team develop their own working methods without prescriptive processes

PLAN-DRIVEN VS. AGILE — THE FUNDAMENTAL DIFFERENCE

Plan-Driven:

`Requirements → [Requirements Spec Document] → Design & Implementation`

(Iteration happens WITHIN activities; formal documents communicate BETWEEN stages)

Agile:

`Requirements Engineering ↔ Design & Implementation (continuously)`

(Iteration happens ACROSS activities; requirements and design evolve TOGETHER)

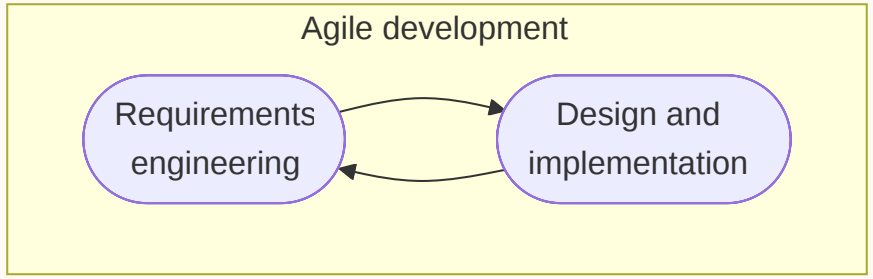
Important nuance: In practice, the boundary is not absolute. Plan-driven processes often incorporate agile practices, and agile methods may include some planned activities. A pure agile process is not inevitably code-only — it may produce design documentation and models when the team decides an iteration should focus on that.

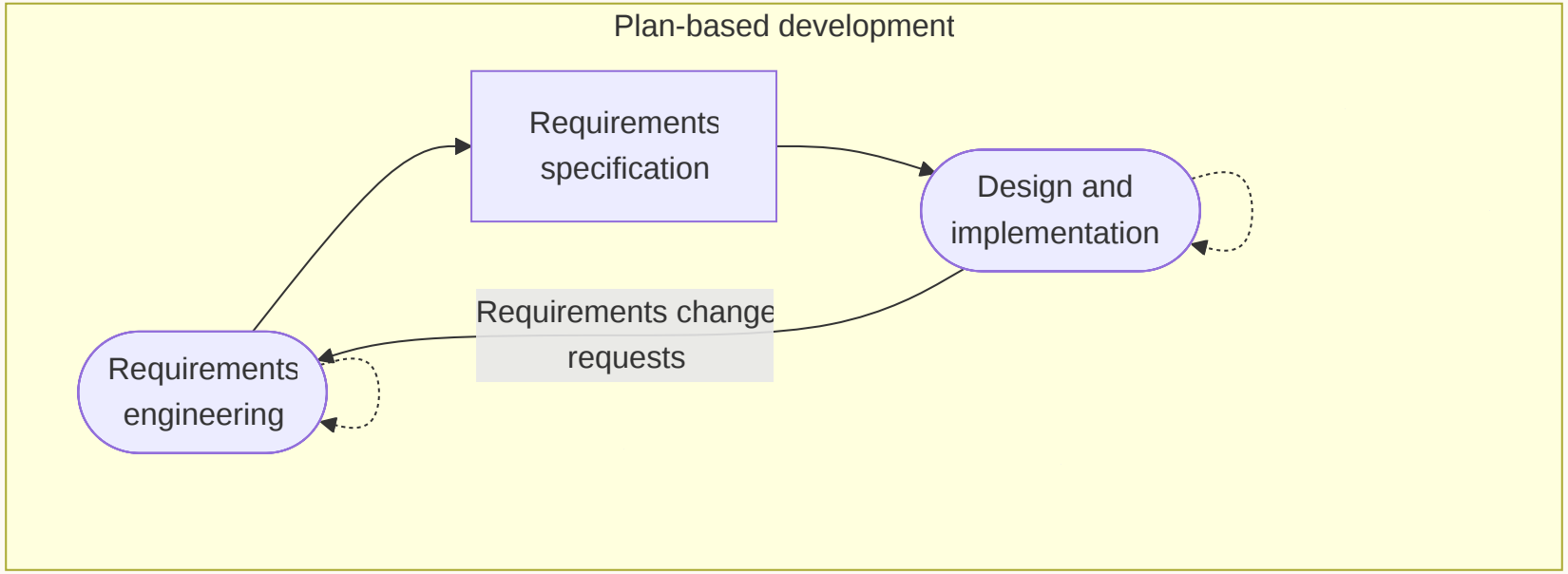
WHERE AGILE WORKS BEST

Two primary domains:

1. **Software product development** — small/medium products for sale (virtually all software products and apps are now developed with agile)
2. **Custom system development within an organization** — where:
   - Clear customer commitment to involvement exists
   - Few external stakeholders and regulations
   - System is standalone (not tightly integrated with parallel development streams)
   - Small/medium team can be co-located, enabling informal communication

AGILE VS PLAN-BASEN DEVELOPMENT





EXTREME PROGRAMMING (XP)

THE FOUNDATIONAL AGILE METHOD

WHAT XP IS:

- Developed by **Kent Beck** — the name reflects pushing recognized good practices to *"extreme"* levels
- Most significant contribution: the **set of agile development practices** it introduced, even if XP itself is rarely used in its pure form today

THE XP RELEASE CYCLE

Select user stories → Plan release → Break down to tasks → Develop/integrate/test software → Release software → Evaluate system

XP'S 10 KEY PRACTICES

Practice	Description
Collective ownership	All developers work on all code; anyone can change anything; no "islands of expertise"
Continuous integration	Completed tasks integrated into whole system immediately; all unit tests must pass after every integration
Incremental planning	Requirements recorded on "story cards"; stories broken into "tasks"
On-site customer	Customer representative available full-time as part of the development team
Pair programming	Developers work in pairs at one computer, checking each other's work
Refactoring	Continuous code improvement whenever opportunities found — keeps code simple and maintainable
Simple design	Only enough design to meet current requirements — no more
Small releases	Minimal useful functionality first; frequent incremental releases
Sustainable pace	No excessive overtime — it reduces code quality and medium-term productivity
Test-first development	Tests written *before* code; automated unit test frameworks used

AGILE DEVELOPMENT - USER STORIES 1/4

- What they are:
  - Scenarios of use that a system user might experience
  - Written collaboratively by customer and development team on **"story cards"**
  - Brief, narrative, scenario-based descriptions of system needs
- How they're used
- Advantages
- Disadvantages

AGILE DEVELOPMENT - USER STORIES 2/4

- What they are
- How they're used:
  - Team breaks stories into **task cards** (e.g.,
    - "Change dose"
    - "Formulary selection"
    - "Dose checking")
  - Estimates effort per task; customer **prioritizes** stories
  - Aim: implement functionality deliverable in ~**2 weeks** per release

AGILE DEVELOPMENT - USER STORIES 3/4

- What they are
- How they're used
- Advantages:
  - Much easier for users to relate to than formal requirements documents or use cases
  - Useful for involving users in requirements elicitation
  - Flexible — unimplemented stories can be changed or discarded as requirements evolve

AGILE DEVELOPMENT - USER STORIES 4/4

- What they are
- How they're used
- Advantages
- Disadvantages
  - **Completeness problem**: Hard to judge if enough stories cover all essential requirements
  - A single story may not give a true picture of an activity
  - Experienced users often omit things they take for granted

REFACTORING

Traditional approach: Design for change — anticipate future changes upfront

XP's rejection of this: Designing for anticipated change is often **wasted effort** — anticipated changes frequently never materialize, and actual changes requested are completely different

What refactoring is:

Why it's necessary:

Practical limitations:

REFACTORING 1/4

- **Traditional approach:**
  - Design for change — anticipate future changes upfront
- **XP's rejection of this:**
  - Designing for anticipated change is often **wasted effort** — anticipated changes frequently never materialize, and actual changes requested are completely different
- What refactoring is
- Why it's necessary
- Practical limitations

REFACTORING 2/4

- What refactoring is:
  - Continuously looking for and implementing improvements to existing code
  - Making improvements **even when there's no immediate need** for them
  - Examples: reorganizing class hierarchies to remove duplicate code, renaming attributes/methods for clarity, replacing similar code sections with library method calls
- Why it's necessary
- Practical limitations

REFACTORING 3/4

- What refactoring is
- Why it's necessary:
  - Incremental development naturally causes **local changes to degrade software structure**
  - Without refactoring: code duplication, inappropriate reuse, structural degradation, workarounds
  - Refactoring maintains **readability and understandability** over time
- Practical limitations

REFACTORING 4/4

- What refactoring is
- Why it's necessary
- Practical limitations:
  - Development pressure causes refactoring to be **delayed** in favor of new features
  - Some changes require **architectural** modification — not addressable by code-level refactoring
  - Development environments include refactoring tools that simplify finding dependencies and making global modifications

TEST-FIRST / TEST-DRIVEN DEVELOPMENT (TFD/TDD) 1/5

- The core innovation:
  - Write tests **before** you write the code — the opposite of traditional "code then test"
  - Tests can be run during coding, discovering problems **during development** not after
- Key features of XP testing
- Why test-first works
- The link to user stories
- Test automation requirements
- Problems with test-first development

TEST-FIRST / TEST-DRIVEN DEVELOPMENT (TFD/TDD) 2/5

- The core innovation
- Key features of XP testing:
  - Test-first development
  - Incremental test development from scenarios
  - User involvement in test development and validation
  - Automated testing frameworks (e.g., **JUnit** for Java)
- Why test-first works
- The link to user stories
- Test automation requirements
- Problems with test-first development

TEST-FIRST / TEST-DRIVEN DEVELOPMENT (TFD/TDD) 3/5

- The core innovation
- Key features of XP testing
- Why test-first works:
  - Writing tests implicitly defines **both an interface AND a specification of behavior**
  - Reduces requirements/interface misunderstandings
  - Forces task implementers to **thoroughly understand the specification** before coding
  - Clarifies ambiguities and omissions before implementation
  - Prevents **"test-lag"** — where implementation gets ahead of testing and tests get skipped

TEST-FIRST / TEST-DRIVEN DEVELOPMENT (TFD/TDD) 4/5

- The core innovation
- Key features of XP testing
- Why test-first works
- The link to user stories
- Test automation requirements:
  - Story cards → Task cards → One or more unit tests per task → Implementation
- Problems with test-first development

TEST-FIRST / TEST-DRIVEN DEVELOPMENT (TFD/TDD) 5/5

- Problems with test-first development:
  - Programmers prefer programming to testing → they write **incomplete tests**, skip edge cases
  - Some tests are very hard to write incrementally (e.g., complex UI display logic and screen workflows)
  - **Coverage completeness** cannot be guaranteed — large test suite ≠ complete coverage
  - Crucial system parts may remain untested despite the appearance of thoroughness
  - Without test review and additional post-development tests, bugs can ship

PAIR PROGRAMMING 1/3

- The practice:
  - Two programmers sit at **one computer** to develop software
  - Pairs are **dynamic** — they rotate so all team members work with each other
- Three key advantages
- Productivity question (nuanced)

PAIR PROGRAMMING 2/3

- The practice
- Three key advantages:
  - **Collective ownership** — reflects "egoless programming" (1971): software owned by team, not individuals; team collectively resolves problems
  - **Informal code review** — every line of code seen by at least two people; cheaper than formal inspections though probably finds fewer errors
  - **Encourages refactoring** — when one developer refactors, the pair benefits immediately, so there's social support for it

PAIR PROGRAMMING 3/3

- Productivity question (nuanced):
  - Intuitive concern: two people at one computer = half the output
  - productivity comparable to two working independently — because pairs discuss before coding (fewer false starts) and informal inspection reduces debugging time
  - significant productivity **loss** vs. two individuals; quality benefits don't fully compensate
  - reduces project risk when team members leave; implicit knowledge is distributed, not siloed

THANK YOU!